

Theoretical Competitive Programming (TCP): A Tripartite Pedagogical Framework Integrating Formal Mathematics, Hardware-Aware C++, and Differential Testing

Nguyen Quan Ba Hong*

September 11, 2026

Abstract

This manuscript presents the Theoretical Competitive Programming (TCP) framework, which aligns competitive programming (CP) with Theoretical Computer Science (TCS). To address pedagogical concerns regarding student cognitive offloading to generative AI, the TCP framework proposes a tripartite protocol. Under this protocol, algorithmic solutions are initially derived using formal mathematical invariants (e.g., monoids, posets, and max-flow min-cut duality). Subsequently, these abstractions are translated into hardware-aware systems-level C++ implementations optimized for memory hierarchy and spatial cache locality. Finally, implementation correctness is evaluated using Python-based differential testing and symbolic verification. By integrating discrete mathematics, systems-level engineering, and automated testing, this methodology structures the use of AI as an auxiliary tool, emphasizing algorithmic reasoning and formal proofs in computer science education.

Contents

1	Introduction	2
2	Literature review and theoretical framework	2
2.1	Methodology of competitive programming	2
2.2	Generative AI and cognitive offloading in CS education	2
3	A TCP tripartite protocol	2
3.1	Phase 1: mathematical theory and invariants	2
3.2	Phase 2: systems-level C++ (hardware considerations) [Str13]	3
3.3	Phase 3: Python differential testing and symbolic verification	3
4	Curricular synthesis of the TCP guidebook	4
5	Implementation and expected impacts	5
6	Conclusion	5

*University of Management and Technology, Ho Chi Minh City, No. 2, 60CL Street, Quarter 9, Cat Lai Ward, Thu Duc City, Ho Chi Minh City 700000, Vietnam
E-mail: hong.nguyenquanba@umt.edu.vn, nguyentuanbahong@gmail.com

1 Introduction

Competitive Programming (CP) has historically served as an established environment for developing algorithmic problem-solving skills [Win06], influencing evaluation methods in technology sectors. However, traditional CP pedagogy often emphasizes rapid coding and pattern recognition over formalisms of Theoretical Computer Science (TCS). Furthermore, the adoption of large language models (LLMs) has introduced challenges to algorithmic education. When confronted with complex algorithmic tasks, students may engage in cognitive offloading [RK02], utilizing generative AI to synthesize code rather than deriving the underlying logic independently.

To address these issues, this manuscript presents the Theoretical Competitive Programming (TCP) framework. Based on an 18-module curriculum, the TCP methodology outlines a tripartite protocol, which integrates formal mathematical analysis with systems-level programming and automated verification [Tao26; Tao06].

2 Literature review and theoretical framework

This section contextualizes the TCP framework by contrasting competitive programming evaluation methods with formal theoretical computer science, and by examining the pedagogical impact of generative AI on algorithmic education.

2.1 Methodology of competitive programming

Traditional competitive programming platforms, such as VNOI, Codeforces, CodeChef, AtCoder, Topcoder, SPOJ (Sphere Online Judge), and the CSES Problem Set, prioritize input-output validation over structural algorithmic proofs. While this encourages rapid prototyping, it permits submissions that may lack formal mathematical justification or tight asymptotic bounds. In contrast, formal theoretical computer science focuses on state-space analysis, invariants, and asymptotic complexity (e.g., distinguishing between deterministic polynomial time P and nondeterministic polynomial time NP). The TCP framework argues that treating CP solely as a coding exercise isolates it from its mathematical foundations.

2.2 Generative AI and cognitive offloading in CS education

The integration of generative AI into computer science education has altered the acquisition of algorithmic reasoning. In traditional pedagogy, “productive struggle” is often used to teach concepts such as dynamic programming state transitions or graph-theoretic topological sorts. Large language models bypass this process by generating syntactically correct solutions. However, these models operate probabilistically, sometimes generating algorithms that are logically flawed or asymptotically suboptimal (e.g., failing to exploit a structural invariant that reduces an exhaustive permutation search of tight complexity $\Theta(n!)$ to an optimal comparison-based sorting procedure with tight asymptotic complexity $\Theta(n \log n)$, where $n \in \mathbb{Z}_{\geq 1}$ denotes the input cardinality).

3 A TCP tripartite protocol

Designed to counter over-reliance on generative AI and reinforce foundational algorithmic reasoning, the TCP framework formalizes a Tripartite Protocol. Each curricular module is structured systematically around this three-phase protocol:

3.1 Phase 1: mathematical theory and invariants

Each computational problem is initially formalized by identifying the governing algebraic structures, topological invariants, and analytical bounds. Across all modules, for any integer $n \in \mathbb{Z}_{\geq 1}$, we define the canonical index sets $[n] := \{1, \dots, n\}$ and $[n]_0 := [n] \cup \{0\}$.

- *Algebraic formulation:* Mapping state transitions to abstract algebraic structures. For instance, segment trees are generalized over any monoid $(S, \oplus, e)$, where $\oplus : S \times S \rightarrow S$ is an associative binary operation and $e \in S$ is the two-sided identity ($\forall a, b, c \in S, (a \oplus b) \oplus c = a \oplus (b \oplus c)$ and $a \oplus e = e \oplus a = a$). Sequence partitioning problems are resolved by applying order-theoretic minimax duality on finite partially ordered sets (posets)

$(P, \leq)$ (with $|P| < \infty$): Dilworth’s theorem equates the poset width to the minimum cardinality of a chain partition,

$$\text{width}(P) = \max\{|A| : A \subseteq P \text{ is an antichain}\} = \min\{|\mathcal{C}| : \mathcal{C} \text{ is a chain partition of } P\}, \quad (1)$$

while Mirsky’s theorem dually equates the poset height to the minimum cardinality of an antichain partition,

$$\text{height}(P) = \max\{|\mathcal{C}| : \mathcal{C} \subseteq P \text{ is a chain}\} = \min\{|\mathcal{A}| : \mathcal{A} \text{ is an antichain partition of } P\}. \quad (2)$$

- *Asymptotic calculus*: Deriving asymptotic bounds by proving that execution time $T : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ and reference complexity $f : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ satisfy the Bachmann–Landau definition:

$$T \in \Theta(f) \iff \exists c_1, c_2 \in \mathbb{R}_{>0}, \exists n_0 \in \mathbb{N} \text{ such that } \forall n \geq n_0, 0 \leq c_1 f(n) \leq T(n) \leq c_2 f(n). \quad (3)$$

This establishes asymptotic bounds for recurrence relations under the Random Access Machine (RAM) model [Knu97].

3.2 Phase 2: systems-level C++ (hardware considerations) [Str13]

While asymptotic bounds characterize mathematical efficiency on abstract machines, empirical execution speed depends on hardware hierarchy and memory architectures.

- *Cache optimization*: Under the Aggarwal–Vitter (M, B) external-memory model [AV88], memory capacity M and block transfer size B satisfy $M, B \in \mathbb{Z}_{\geq 1}$ with $B \in [M/2]$. In the external regime where the problem size satisfies $n > M$ (implying $n \geq 3$), I/O complexity $Q(n; M, B)$ measures block transfers between internal cache and secondary memory. Contiguous flat-array allocations achieve the sequential scanning bound

$$\text{Scan}(n) = \Theta\left(\left\lceil \frac{n}{B} \right\rceil\right) \quad (4)$$

transfers, mitigating the worst-case $\Omega(n)$ cache misses characteristic of pointer-based node representations.

- *Bitwise and memory constraints*: Representing subset state spaces as bit vectors in $\{0, 1\}^n$ enables compact memory layouts and word-level bitwise operations, bridging hardware caches to subset computations. Over the Boolean lattice $\mathcal{B}_n := (\mathcal{P}([n]), \subseteq)$ for any function $F : \mathcal{P}([n]) \rightarrow R$ taking values in a commutative ring $(R, +, \cdot)$, the sub-lattice summation

$$\hat{F}(U) = \sum_{V \subseteq U} F(V), \quad \forall U \in \mathcal{P}([n]), \quad (5)$$

is computed via the Fast Zeta Transform (FZT) in exactly $n2^{n-1}$ scalar additions (tight asymptotic complexity $\Theta(n2^n)$ additions, compared to $\Theta(3^n)$ for naive submask iteration and $\Theta(n^2 2^n)$ for ranked subset convolutions). Processing transitions dimension-by-dimension over contiguous flat arrays yields sequential memory access patterns that enhance spatial cache locality and eliminate pointer indirection overhead.

3.3 Phase 3: Python differential testing and symbolic verification

To verify implementation correctness against systems-level bugs and integer edge cases, differential testing harnesses and symbolic reference oracles are constructed in Python, decoupling verification from hardware-level execution constraints.

- *Test generation*: Automated test generators construct boundary cases and stress inputs (e.g., degenerate topologies, dense bipartite graphs, extremal integer boundaries).
- *Symbolic computation*: Combining Python’s native arbitrary-precision integer arithmetic (preventing fixed-width integer overflow) with computer algebra systems such as SymPy [Meu+17] (performing exact symbolic derivations to eliminate floating-point rounding errors) and multidimensional array structures via NumPy [Har+20], reference solvers compute exact combinatorial quantities.

- *Differential testing*: The output of the optimized implementation ($\mathcal{S}_{\text{opt}}$) is empirically compared against the reference combinatorial oracle ($\mathcal{S}_{\text{ref}}$) across a sampled distribution of adversarial topologies $\mathcal{D}$. Evaluating $m \in \mathbb{Z}_{\geq 1}$ independent and identically distributed (i.i.d.) test instances drawn from $\mathcal{D}$ with zero observed discrepancies ensures, via $(1 - \epsilon)^m < e^{-\epsilon m} \leq \delta$, with statistical confidence at least $1 - \delta$ that the failure probability satisfies $\mathbb{P}_{x \sim \mathcal{D}}[\mathcal{S}_{\text{opt}}(x) \neq \mathcal{S}_{\text{ref}}(x)] \leq \epsilon$ whenever

$$m \geq \left\lceil \frac{1}{\epsilon} \ln \frac{1}{\delta} \right\rceil \quad (6)$$

(with exact discrete threshold $\lceil \frac{\ln \delta}{\ln(1-\epsilon)} \rceil$), for any tolerance parameters $\epsilon, \delta \in (0, 1)$. On finite bounded subdomains $\mathcal{D}_{\leq n_0}$, exhaustive enumeration provides deterministic verification: $\forall x \in \mathcal{D}_{\leq n_0}, \mathcal{S}_{\text{opt}}(x) = \mathcal{S}_{\text{ref}}(x)$.

4 Curricular synthesis of the TCP guidebook

The TCP Guidebook applies this protocol across an 18-module curriculum, spanning foundational discrete algorithms to computational complexity and NP-completeness. Representative curriculum components include:

1. *Advanced data structures (rollback DSU)*: Formalizing the Rollback Disjoint-Set Union (DSU) over an n -element universe ($n \in \mathbb{Z}_{\geq 1}$) by omitting path compression and maintaining union-by-size. Because any component tree $\mathcal{T}$ of height h satisfies $|V(\mathcal{T})| \geq 2^h$, the tree height satisfies the structural bound $2^{\text{height}(\mathcal{T})} \leq |V(\mathcal{T})| \leq n$, implying

$$\text{height}(\mathcal{T}) \leq \lfloor \log_2 n \rfloor, \quad (7)$$

establishing an $\mathcal{O}(\log n)$ worst-case query time. Recording the state mutations (the parent link and component size) per union on an execution stack permits state restoration in $\Theta(1)$ time per rollback operation.

2. **Graph topology and flow networks**: Modeling flow networks $G = (V, E, c)$ with non-negative rational edge capacities $c : E \rightarrow \mathbb{Q}_{\geq 0}$ (or integer capacities $c : E \rightarrow \mathbb{Z}_{\geq 0}$), where the Max-Flow Min-Cut theorem establishes primal-dual optimality. Monotonicity of shortest residual distances $d_{k+1}(s, t) \geq d_k(s, t) + 1$ bounds Dinic's algorithm to at most $|V| - 1$ layered phases, yielding a worst-case time complexity of $\mathcal{O}(|V|^2|E|)$. On unit networks with unit vertex capacities ($c(v) = 1$ for all $v \in V \setminus \{s, t\}$), the phase count is bounded by $2\sqrt{|V|}$ via the Even-Tarjan theorem, reducing the bound to

$$T_{\text{unit}}(V, E) = \mathcal{O}(|E|\sqrt{|V|}) \quad (8)$$

(as applied in maximum bipartite matching).

3. *String combinatorics (suffix arrays and LCP)*: Constructing Suffix Arrays for a string s of length $n \in \mathbb{Z}_{\geq 1}$ over an alphabet Σ in $\mathcal{O}(n \log n)$ time via prefix-doubling equivalence ranking, and Longest Common Prefix (LCP) arrays in linear time $\mathcal{O}(n)$ via Kasai's algorithm. For all $i \in [n - 1]$ satisfying $\text{rank}[i + 1] > 1$ (with $h[i]$ denoting the LCP of suffix i with its lexicographical predecessor suffix in the suffix array, and $h[k] := 0$ when $\text{rank}[k] = 1$), Kasai's algorithm exploits the positional invariant

$$h[i + 1] \geq h[i] - 1, \quad (9)$$

bounding the cumulative character comparison increments across all iterations by at most $2n$.

4. *Combinatorial group theory*: For a finite symmetry group G acting on a finite set X , Burnside's Lemma computes the orbit count

$$|X/G| = \frac{1}{|G|} \sum_{g \in G} |X^g|, \quad (10)$$

where $X^g := \{x \in X : g \cdot x = x\}$ is the fixed-point set under $g \in G$. When G acts on a finite domain D and $X = C^D$ denotes the set of colorings with a palette C of $k = |C|$ colors, the Pólya Enumeration Theorem evaluates the cycle index polynomial $Z_G(t_1, \dots, t_{|D|})$, specializing to $|C^D/G| = Z_G(k, \dots, k) = \frac{1}{|G|} \sum_{g \in G} k^{c(g)}$ (where $c(g)$ is the number of disjoint cycles of g on D), enumerating non-isomorphic equivalence classes without exhaustive state-space search.

5. *Complexity theory (P vs NP)*: Formalizing computational limits through Karp’s polynomial-time many-one reductions between formal languages $L_1, L_2 \subseteq \Sigma^*$ over a finite alphabet Σ :

$$L_1 \leq_p L_2 \iff \exists \text{ polynomial-time computable function } f : \Sigma^* \rightarrow \Sigma^* \text{ such that } \forall x \in \Sigma^*, x \in L_1 \iff f(x) \in L_2. \quad (11)$$

The execution time $T_f(|x|) = \mathcal{O}(|x|^k)$ for some constant $k \geq 1$ upper-bounds the output length $|f(x)| \leq T_f(|x|)$, ensuring that the composition of reductions is polynomial-time computable and establishing the theoretical boundaries of computational tractability.

5 Implementation and expected impacts

Implementing the TCP methodology provides an alternative assessment model for undergraduate and competitive algorithms curricula. By structuring coursework across mathematical invariants, systems-level implementations, and differential testbenches, the framework evaluates formal reasoning, architectural awareness, and verification rigor concurrently. Within this pedagogical workflow, generative AI is framed as an auxiliary tool for exploratory syntax generation and debugging rather than as an authoritative substitute for algorithmic design. Although the framework is designed to counter passive reliance on automated code synthesis, claims regarding its empirical efficacy in mitigating cognitive offloading remain hypotheses to be evaluated through controlled educational cohort studies, rather than established experimental findings.

6 Conclusion

The Theoretical Competitive Programming (TCP) framework provides a formal pedagogical structure for computer science education. By applying the tripartite protocol, it integrates mathematical derivation, systems-level software engineering, and automated verification into a coherent problem-solving framework. Future empirical research will examine the deployment of this curriculum across undergraduate student cohorts to quantitatively evaluate educational outcomes and assess its influence on student problem-solving strategies and cognitive offloading tendencies.

Data and code availability

No empirical datasets involving human participants or student cohorts were generated or analyzed during this theoretical and methodological study. To facilitate independent verification and educational adaptation, the reference algorithmic implementations (in C++ and Python), hardware benchmark scripts, automated differential test harnesses, software environment specifications, and the companion Vietnamese TCP Guidebook are available at https://github.com/NQBH/theoretical_competitive_programming.

Acknowledgment

The author acknowledges the institutional and infrastructural support provided by the University of Management and Technology, Ho Chi Minh City (UMT HCMC). Gratitude is extended to the administration and the School of Technology at UMT HCMC for their support during the formalization and verification of this theoretical framework.

References

- [AV88] Alok Aggarwal and Jeffrey Scott Vitter. “The Input/Output Complexity of Sorting and Related Problems”. In: *Communications of the ACM* 31.9 (1988), pp. 1116–1127.
- [Har+20] Charles R Harris, K Jarrod Millman, Stéfan J Van Der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J Smith, et al. “Array programming with NumPy”. In: *Nature* 585.7825 (2020), pp. 357–362.
- [Knu97] Donald Ervin Knuth. *The art of computer programming*. Pearson Education, 1997.
- [Meu+17] Aaron Meurer et al. “SymPy: symbolic computing in Python”. In: *PeerJ Computer Science* 3 (2017), e103.

- [RK02] Leonid Rozenblit and Frank Keil. “The misunderstood limits of folk science: An illusion of explanatory depth”. In: *Cognitive science* 26.5 (2002), pp. 521–562.
- [Str13] Bjarne Stroustrup. *The C++ programming language*. 4th. Pearson Education, 2013.
- [Tao06] Terence Tao. *Solving Mathematical Problems: A Personal Perspective*. Oxford University Press, 2006.
- [Tao26] Terence Tao. “Mathematics in the Age of AI”. In: *arXiv preprint arXiv:2608.16753* (2026).
- [Win06] Jeannette M Wing. “Computational thinking”. In: *Communications of the ACM* 49.3 (2006), pp. 33–35.